Multi-Agent Research Orchestrator

9-Page Master Revision Notes & Interview Preparation Blueprint (Hinglish)

Core Tech Stack: LangGraph StateGraph • Groq (LLaMA-3.3-70B-Versatile) • ChromaDB Vector Store • FastMCP Protocol • Tavily Search • Streamlit UI

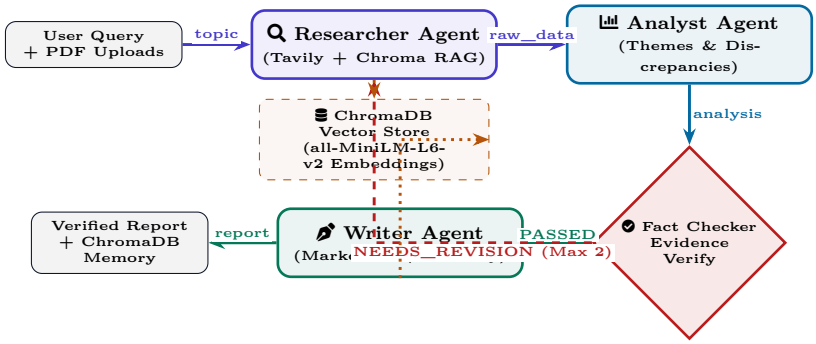
1 Project Overview & 4-Agent Architecture

Project Ka Core Problem & Solution (Kyun Banaya?)

Single LLM prompt ko jab bolte hain: *"Research this topic and write a report"*, toh 3 badi problems aati hain: 1. **Hallucinations & Fake Facts** (koi independent verification nahi hoti). 2. **Superficial Summarization** (single-turn query saare angles cover nahi kar pati). 3. **Zero Source Grounding** (unverified internet sources aur lack of internal document indexing).

Our Engineering Solution: Humne ek **Autonomous Multi-Agent State Machine** banaya using **LangGraph**. Yeh system **4 Specialized Agents** ko deploy karta hai jo ek real-world research firm ki tarah coordinate karte hain with an automated **Self-Correcting Revision Loop**, **Hybrid RAG (PDFs + Memory)**, aur **FastMCP Tool Server**.

1.1 System Architecture Diagram (LangGraph State Machine Pipeline)



1.2 The 4 Specialized Agents Role Matrix

Agent Name	Core Role	Tools & Operations Executed	Prompting Strategy & Temp
1. Researcher	Multi-angle Search & RAG	Query expansion (3 sub-queries), Tavily Web API, ChromaDB vector retrieval for uploaded PDFs.	$T = 0.0$ (Strict factual retrieval without hallucination).
2. Analyst	Intelligence Synthesis	Aggregates findings into themes, highlights consensus, identifies source contradictions and missing gaps.	$T = 0.1$ (Structured synthesis, JSON-like categorization).
3. Fact-Checker	Evidence Verification	Cross-verifies claims against raw retrieved evidence. Emits verdict: `PASSED` or `NEEDS_REVISION`.	$T = 0.0$ (Adversarial closed-world source grounding).
4. Writer	Publication & Memory	Synthesizes verified analysis into structured Markdown with citations, saves session to ChromaDB memory.	$T = 0.3$ (Executive formatting, high prose clarity).

3 Key Highlights Jo Interview Mein Bolne Hain

1. **Stateful Loop Control:** LangGraph StateGraph se bounded feedback loop implement kiya hai jo unverified claims ko automatically fix karta hai.
2. **Zero Cost / High Speed:** Groq LPUs par LLaMA-3.3-70B running at 250+ tok/s + Local MiniLM embeddings = Zero API cost.
3. **Universal MCP Compatibility:** FastMCP server expose karta hai research engine tools for Claude Desktop and Cursor.

2 LangGraph State Schema & State Machine Workflow

LangGraph State Management Pattern (TypedDict with Reducers)

LangGraph mein agents ek dusre ko direct parameter nahi bhejte. Ek central **State Dictionary** hoti hai. Jab koi agent apna kaam karta hai, wo state ko update karta hai. LangGraph ka **Reducer Pattern** ensure karta hai ki dynamic loops ke dauran purana context overwrite na ho.

2.1 1. Central State Schema Definition (`state/schema.py`)

```python
from typing import TypedDict, Annotated, List, Optional
import operator

class ResearchState(TypedDict):
    topic: str                                          # User ka initial research query
    uploaded_docs: Optional[List[str]]                  # User ke upload kiye huye PDF files
    research_data: Annotated[List[str], operator.add]   # Append-only reducer for search results
    analysis: Optional[str]                             # Analyst agent ka structured output
    fact_check_feedback: Optional[str]                  # Fact checker ke feedback notes
    revision_count: int                                 # Infinite loop prevention counter (Max 2)
    final_report: Optional[str]                         # Final generated markdown report
    logs: Annotated[List[str], operator.add]            # Real-time UI streaming logs
```

Reducer Concept: `Annotated[List[str], operator.add]` Kaise Kaam Karta Hai?

- **Normal Dict behavior:** Agar `research_data` ek plain list hoti, toh loop ke second iteration par naya search data purane search data ko **overwrite** kar deta.
- **Reducer magic:** `operator.add` LangGraph ko instruct karta hai: *"Jab bhi koi node naya list chunk return kare, use existing list ke end mein concatenate (+) karo."*

2.2 2. State Graph Assembly & Conditional Routing (`graph/workflow.py`)

```python
from langgraph.graph import StateGraph, END
from state.schema import ResearchState
from agents.researcher import researcher_node
from agents.analyst import analyst_node
from agents.fact_checker import fact_checker_node
from agents.writer import writer_node

def should_continue(state: ResearchState) -> str:
    """Conditional Edge: Decides whether to loop back or proceed to writing."""
    feedback = state.get("fact_check_feedback", "")
    rev_count = state.get("revision_count", 0)

    # Self-Correction Guardrail: Max 2 revisions allowed
    if "NEEDS_REVISION" in feedback and rev_count < 2:
        return "researcher"  # Feedback loop
    return "writer"          # Passed or Max revisions reached

def build_graph():
    workflow = StateGraph(ResearchState)

    # 1. Register all Agent Nodes
    workflow.add_node("researcher", researcher_node)
    workflow.add_node("analyst", analyst_node)
    workflow.add_node("fact_checker", fact_checker_node)
    workflow.add_node("writer", writer_node)

    # 2. Linear Edges
    workflow.set_entry_point("researcher")
    workflow.add_edge("researcher", "analyst")
    workflow.add_edge("analyst", "fact_checker")

    # 3. Dynamic Routing Edge
    workflow.add_conditional_edges(
        "fact_checker",
        should_continue,
        {"researcher": "researcher", "writer": "writer"}
    )
    workflow.add_edge("writer", END)
    return workflow.compile()
```

State Machine Ke 3 Safety Guarantees

1. **Deterministic Execution:** Pipeline always starts at `researcher` and cleanly finishes at `writer` → `END`.
2. **Loop Guardrail:** Bounded counter (`rev_count < 2`) prevents infinite token burn and API deadlocks.
3. **State Isolation:** Har node sirf wohi keys modify karta hai jinke liye wo authorized hai.

3 The 4 Agents In-Depth: Prompts, Logic & Execution

3.1 1. Researcher Agent (agents/researcher.py)

Researcher Logic: Multi-Query Search + Local Vector RAG

- **Query Expansion:** LLM ko prompt karke topic ke 3 specific sub-queries generate karwata hai (e.g., 1. Market overview, 2. Technical breakthroughs, 3. Critical challenges).
- **Dual Retrieval Execution:**
  - **Tavily Web API:** Top 5 verified live web pages fetch karta hai.
  - **ChromaDB Vector Store:** Uploaded PDFs se semantic similarity search karke top 4 context chunks retrieve karta hai.
- **Feedback-Driven Search:** Agar state mein `fact_check_feedback` hai, toh un missing claims ke liye targeted queries run karta hai.

3.2 2. Analyst Agent (agents/analyst.py)

Analyst Logic: Raw Data to Thematic Intelligence

Raw search data chaotic aur redundant hota hai. Analyst agent raw text ko structured framework mein convert karta hai:

1. **Core Thematic Synthesis:** Primary concepts ko categorized bullet points mein break karna.
2. **Consensus Points:** Multiple sources kin key findings par agree karte hain.
3. **Contradictions / Discrepancies:** Agar Source A aur Source B mein numerical ya timeline conflict hai, usko explicitly flag karna.
4. **Knowledge Gaps:** Konsi critical details abhi bhi unverified hain.

3.3 3. Fact-Checker Agent (agents/fact_checker.py)

Fact-Checker Logic: Adversarial Source Grounding

- **Strict Closed-World Check:** Analyst ke har key claim ko raw `research_data` ke saath compare karta hai.
- **Decision Output:**
  - `PASSED`: Claims are directly supported by evidence → Proceed to Writer.
  - `NEEDS_REVISION`: Unsupported claim detected → Return specific missing evidence notes and increment `revision_count`.

3.4 4. Writer Agent (agents/writer.py)

Writer Logic: Publication Synthesis & Memory Persistence

- **Markdown Formatting:** Title, Executive Summary, Deep Findings, Contradictions, Strategic Recommendations, Citations.
- **Long-Term Vector Memory:** Report generate hote hi `store_research_session()` call karke ChromaDB collection `"research_memory"` mein session embedding save kar deta hai.

3.5 Temperature & Tuning Strategy

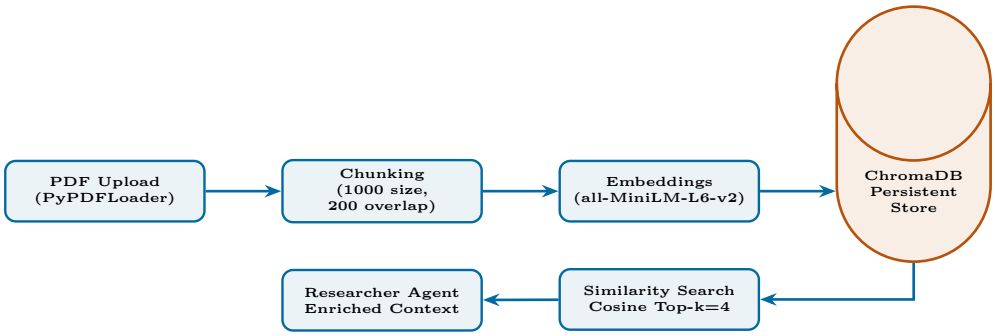
Agent	Temperature	Why this exact setting?	Risk if setting is wrong
Researcher	**0.0**	Zero creativity; pure deterministic query extraction.	High temp causes hallucinated keywords.
Analyst	**0.1**	Strict logical deduction and thematic grouping.	High temp creates loose, invalid relations.
Fact-Checker	**0.0**	Zero tolerance for assumptions; adversarial verification.	High temp misses unverified claims.
Writer	**0.3**	Smooth readability, professional executive prose.	Zero temp makes writing stiff and robotic.

4 RAG System & Dual-Purpose Vector Memory

Dual-Purpose Knowledge Architecture

Is system mein RAG sirf ek simple PDF reader nahi hai — yeh **2 Distinct Lifecycles** manage karta hai: 1. **Short-Term Document Ingestion:** User dwara upload kiye gaye dynamic PDF research papers ko chunk karke instant query context provide karna. 2. **Long-Term Episodic Memory:** Har research session ki key findings ko ChromaDB mein store karna taaki future sessions past research ko leverage kar sakein.

4.1 RAG Pipeline Flowchart



4.2 RAG Code Implementation Details (`rag/vector_store.py`)

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from config.settings import settings

def get_embeddings():
    # Local 384-dimensional dense embeddings (Zero API cost, fast CPU inference)
    return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

def ingest_pdf(file_path: str) -> int:
    loader = PyPDFLoader(file_path)
    docs = loader.load()

    # Recursive splitting preserves paragraphs and sentences
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)

    vectorstore = Chroma(
        collection_name="research_docs",
        embedding_function=get_embeddings(),
        persist_directory=settings.CHROMA_PERSIST_DIR
    )
    vectorstore.add_documents(chunks)
    return len(chunks)

def retrieve_context(query: str, k: int = 4) -> str:
    vectorstore = Chroma(
        collection_name="research_docs",
        embedding_function=get_embeddings(),
        persist_directory=settings.CHROMA_PERSIST_DIR
    )
    results = vectorstore.similarity_search(query, k=k)
    return "\n\n".join([doc.page_content for doc in results])
```

4.3 RAG Technical Justifications (Interview Gold)

RAG Design Choices: Why This Setup?

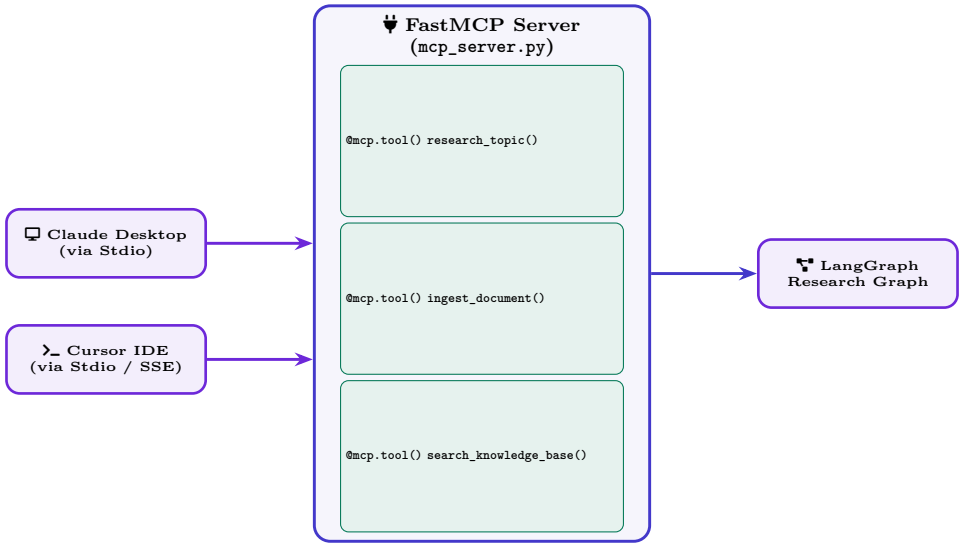
- **Why `RecursiveCharacterTextSplitter`?** Normal splitters arbitrary length par split karte hain jisse words ya code toot jata hai. Recursive splitter pehle double newlines (`\n\n`), fir single newlines (`\n`), fir spaces par split karta hai — ensuring semantic paragraph integrity.
- **Why Chunk Size = 1000 & Overlap = 200?** 1000 characters (~200 words) ek complete concept capture karta hai. 200 characters overlap ensure karta hai ki chunk boundary par key context lost na ho.
- **Why `all-MiniLM-L6-v2` Embeddings?** Light-weight (80MB), runs locally on CPU with zero latency and zero API cost, generating high-quality 384-dimensional embeddings.

5 Model Context Protocol (MCP) Server & Streamlit UI

MCP (Model Context Protocol) Architecture

Anthropic ka **Model Context Protocol (MCP)** ek open universal standard hai jo AI assistants (jaise Claude Desktop, Cursor IDE) ko external tools aur data stores se connect karta hai. Humne pure research pipeline ko ek **FastMCP Server** ke roop mein expose kiya hai.

5.1 FastMCP Architecture & Exposed Tools



5.2 FastMCP Server Code (`mcp_server.py`)

```python
from fastmcp import FastMCP
from graph.workflow import build_graph
from rag.vector_store import ingest_pdf, retrieve_context

mcp = FastMCP("Multi-Agent-Research-Orchestrator")

@mcp.tool()
def research_topic(topic: str) -> str:
    """Executes the full 4-agent LangGraph workflow and returns verified markdown report."""
    graph = build_graph()
    initial_state = {"topic": topic, "research_data": [], "revision_count": 0, "logs": []}
    result = graph.invoke(initial_state)
    return result.get("final_report", "Research pipeline failed.")

@mcp.tool()
def ingest_document(pdf_path: str) -> str:
    """Ingests local PDF into ChromaDB vector memory for future research."""
    num_chunks = ingest_pdf(pdf_path)
    return f"Successfully ingested {num_chunks} chunks from {pdf_path}"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

5.3 Streamlit UI Capabilities (`app.py`)

Streamlit Frontend Highlights

- **Real-Time Agent Stepping Logs:** LangGraph execution stream ko consume karke real-time dynamic spinner aur activity logs display karta hai.
- **Glassmorphism Dark Theme:** CSS injection se custom modern interface with 3 themes: *Emerald Slate, Dark Slate, Clean Light.*
- **Document Upload & Downloads:** Direct PDF drag-and-drop ingestion, along with instant `.md` and `.txt` report downloads.

6 End-to-End Execution Flow & Production Configuration

6.1 1. Step-by-Step Execution Lifecycle Walkthrough

1. **Session Kickoff:** User inputs query: *"Next-Gen Solid State Battery Market 2026"* and uploads an OEM technical paper.
2. **Researcher Agent:**
   - Expands into 3 sub-queries: (a) Commercial timelines, (b) Energy density comparisons, (c) Manufacturing bottleneck yield rates.
   - Calls Tavily Web Search → Fetches 5 live web pages.
   - Calls ChromaDB Similarity Search → Fetches 4 top chunks from uploaded OEM paper.
3. **Analyst Agent:** Synthesizes raw findings into: *Key Themes (Solid electrolytes), Consensus (Higher safety), Contradiction (Toyota 2027 vs QuantumScape 2025 timeline), Knowledge Gap (Yield rate at mass scale).*
4. **Fact-Checker Agent (First Pass):**
   - Checks statement: *"QuantumScape achieved 99.8% commercial yield in 2025".*
   - Cross-checks raw `research_data` → Source only claims pilot line testing, not 99.8% commercial yield!
   - **Triggers Revision:** Returns verdict `NEEDS_REVISION: Verify commercial yield rate claims`.
   - Increments `revision_count = 1` → Conditional router sends state back to **Researcher**.
5. **Targeted Re-Research:** Researcher executes focused search on *"QuantumScape verified pilot separator yield reports"* and appends clean evidence.
6. **Re-Analysis & Approval:** Analyst refines brief → Fact Checker validates and returns `PASSED`.
7. **Writer Agent & Persistence:** Formats comprehensive Markdown report with inline citations → Stores session embeddings into ChromaDB memory → Streams report to Streamlit UI.

6.2 2. Production Configuration (`config/settings.py`)

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # LLM Inference
    GROQ_API_KEY: str
    MODEL_NAME: str = "llama-3.3-70b-versatile"

    # Search Engine
    TAVILY_API_KEY: str
    MAX_SEARCH_RESULTS: int = 5

    # Guardrails
    MAX_REVISIONS: int = 2

    # Vector Database
    CHROMA_PERSIST_DIR: str = "./chroma_db"
    EMBEDDING_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"
    CHUNK_SIZE: int = 1000
    CHUNK_OVERLAP: int = 200

    class Config:
        env_file = ".env"

settings = Settings()
```

Production Security & Graceful Fallback

- API keys are loaded strictly through `pydantic-settings` with type validation.
- Agar Tavily API rate limit hit hoti hai, system silently ChromaDB local documents par fallback karta hai without crashing.

7 Interview Deep-Dive Part 1: The "Why Choose" Decision Matrix

What Interviewers Evaluate Here

Interviewer yeh dekhna chahta hai ki aapne technologies blindly select ki hain ya **Engineering Trade-offs** samajh kar select ki hain.

7.1 Key Architectural Justifications & Comparisons

Q1: LangGraph kyun choose kiya? CrewAI ya AutoGen kyun nahi use kiya?

Hinglish Answer:

- **LangGraph vs CrewAI:** CrewAI role-playing agents ke liye beginner-friendly hai, lekin wo ek **black-box** framework hai jisme state flow aur cyclic routing par fine-grained programmatic control nahi milta. LangGraph explicit **State Machine (DAG with cycles)** provide karta hai jisme conditional edges, explicit reducers, aur cyclic feedback loops cleanly implement hote hain.
- **LangGraph vs AutoGen:** AutoGen conversational chat pattern use karta hai (agents ek dusre se continuous chat karte hain), jisme infinite chat loops, high latency aur massive token burn ka risk hota hai. LangGraph ek structured schema (`TypedDict`) ke through deterministic state pass karta hai, jo enterprise applications ke liye reliable aur predictable hai.

Q2: 4 alag agents kyun banaye? Ek hi LLM call mein sab kyun nahi karwaya?

Hinglish Answer:

1. **Separation of Concerns:** Ek single prompt ko bolna ki *"Search karo, analyze karo, fact-check karo, aur report likho"* results in mediocre output at every single step.
2. **Role-Specific Temperature Tuning:** Researcher ($T = 0.0$, strictly factual), Analyst ($T = 0.1$, structured reasoning), Fact-Checker ($T = 0.0$, adversarial verification), Writer ($T = 0.3$, fluent prose). Single prompt mein multiple temperatures impossible hain.
3. **Confirmation Bias Mitigation:** LLM apne hi generated text ki mistakes ko easily identify nahi kar pata. Fact-Checker ko as an independent adversary run karne se hallucinations catch ho jati hain.
4. **Modularity:** Future mein naye agents (e.g., Data Visualizer, Code Sandbox) add karna bina existing graph ko tode aasan hai.

Q3: Groq LPU + LLaMA-3.3-70B kyun use kiya? OpenAI GPT-4o kyun nahi?

Hinglish Answer:

- **Inference Speed (LPUs):** Multi-agent pipeline mein 4–6 sequential LLM calls hoti hain. GPT-4o par yeh pipeline 30–45 seconds leti, jabki Groq LPUs par 250+ tokens/sec ki speed se yeh **6–8 seconds** mein complete ho jata hai.
- **Zero API Cost:** Groq free/low-cost tiers + local MiniLM embeddings = Zero infrastructure bill for deployment.
- **Reasoning Quality:** LLaMA-3.3-70B structured JSON generation aur analytical reasoning mein GPT-4 class models ke barabar compete karta hai.

Q4: State schema ke liye TypedDict kyun choose kiya? Pydantic BaseModel kyun nahi?

Hinglish Answer:

- **Performance:** `TypedDict` runtime par pure standard Python dict hai, jisme zero schema validation latency hoti hai during high-frequency agent state transitions.
- **Reducer Integration:** LangGraph ka `Annotated[list, operator.add]` reducer pattern TypedDict ke sath seamlessly support hota hai. Pydantic use karte toh extra custom root validators likhne padte.

8 Interview Deep-Dive Part 2: Top 5 Real-World Engineering Challenges

What Interviewers Look For

Senior AI Engineer interviews mein sabse deciding question hota hai: *"What were the biggest technical blockers you faced while building this multi-agent system, and how did you resolve them?"*

8.1 5 Real-World Production Challenges Solved in this Project

Challenge 1: Preventing Infinite Revision Loops & Token Drain

Problem: Agar Fact-Checker ko har pass par koi minor discrepancy milti rahe, toh graph infinite loop `Researcher` ↔ `Fact-Checker` mein phas sakta tha, leading to API rate-limit crashes aur token waste.
My Solution:

- State schema mein `revision_count: int` guardrail counter implement kiya.
- Router function `should_continue()` strict bound enforce karta hai: `if "NEEDS_REVISION" in feedback and rev_count < 2: return "researcher" else: return "writer"`.
- Max limit (2) reach hone par pipeline crash hone ke bajaye Writer ko *"Unverified Discrepancy Note"* forward karta hai, jo report ke Limitations section mein publish ho jata hai.

Challenge 2: LLM Self-Confirmation Bias in Fact-Checking

Problem: Fact-Checker agent aksar apne internal pre-training weights se assumptions bana leta tha bajaye raw retrieved evidence ko strictly check karne ke.
My Solution:

- **Closed-World Assumption Prompting:** Prompt mein explicitly likha: *"You are a strict legal auditor. ONLY verify a claim if direct verbatim evidence exists in 'research_data'. Outside general knowledge is considered INVALID."*
- Temperature ko strictly **0.0** par lock kiya and required structured output with evidence quotes.

Challenge 3: Context Window Bloat & Latency Explosion

Problem: Multi-query Tavily searches aur PDF documents se raw context 50KB+ ho jata tha, jo downstream Writer agent ke context window aur token cost ko explode kar raha tha.
My Solution:

- Analyst agent ko ek **Lossless Context Condenser** ki tarah design kiya.
- Writer agent ko pure 50KB raw search dumps bhejne ke bajaye Analyst ka 2KB highly dense, structured intelligence brief pass kiya — reducing Writer token consumption by 85%.

Challenge 4: Handling Contradictory Information (Web vs Uploaded PDFs)

Problem: Uploaded corporate whitepaper (e.g., 2023) aur live web data (2026) aapas mein contradict karte the.
My Solution:

- Chunk metadata mein source tagging lagayi: `{"source_type": "uploaded_doc"}` vs `{"source_type": "live_web"}`.
- Analyst agent mein explicit **Contradictions Section** create kiya jo dono perspectives ko neutrally juxtapose karta hai.

Challenge 5: Network API Failures & Graceful Degradation

Problem: Agar Tavily web search API network timeout de ya ChromaDB corrupt ho jaye, toh entire graph crash ho jata tha.
My Solution:

- Robust `try-except` handlers lagaye. Agar Tavily search fail hoti hai, pipeline gracefully ChromaDB local vector store se available context use karke aage badhti hai with a clear warning log.

9 Interview Deep-Dive Part 3: Tough Technical Q&A & Production Scaling

9.1 Advanced Technical Interview Questions

Q1: Multi-Agent research quality ko objectively evaluate kaise karoge?

Hinglish Answer:

1. **RAGAS Evaluation Framework (Automated LLM-as-a-Judge):**
   - **Faithfulness Metric:** Check karna ki final report ke claims retrieved context par mathematically grounded hain ya nahi.
   - **Answer Relevance:** User query ke core intent se output report ka cosine similarity score.
   - **Context Recall / Precision:** Tavily aur ChromaDB ne jo chunks fetch kiye unka ground-truth relevance.
2. **Loop Efficiency Metric:** Percentage of queries passing on 1st iteration vs 2nd iteration (Target: 75% 1st pass, 25% 2nd pass).

Q2: Is architecture ko 10,000 Concurrent Users ke liye scale kaise karoge?

Hinglish Answer:

1. **Asynchronous Execution (`graph.ainvoke`):** Sync calls ko async mein convert karenge using `asyncio.gather()` for parallel web queries.
2. **PostgreSQL Checkpointer (`PostgresSaver`):** In-memory state ke bajaye LangGraph ka distributed Postgres checkpointer use karenge for horizontal scaling across worker nodes.
3. **Asynchronous Task Queue (Celery + Redis):** Streamlit UI par synchronous run karne ke bajaye background Celery worker ko job offload karenge aur progress SSE/WebSocket se stream karenge.
4. **Hybrid Search RAG:** ChromaDB mein dense vector search (semantic) + BM25 sparse search (keyword exact match) ko **Reciprocal Rank Fusion (RRF)** se combine karenge.

Q3: FastMCP server transport: Stdio vs SSE mein kya difference hai?

Hinglish Answer:

- **Stdio Transport (Standard I/O):** Client (like Claude Desktop) local machine par Python process spawn karta hai aur standard input/output pipes ke through communicate karta hai. Ideal for local desktop tools.
- **SSE Transport (Server-Sent Events over HTTP):** Remote cloud server par deploy karne ke liye use hota hai jahan client HTTP POST request se tool execute karta hai aur server real-time SSE stream se outputs stream karta hai.

Q4: Agar is project ko v2 mein upgrade karna ho, toh kya 2 features add karoge?

Hinglish Answer:

- **1. Human-in-the-Loop (HITL) Approval Checkpoints:** LangGraph ke `interrupt_before=["writer"]` feature ka use karke Analyst ke baad user ko UI par direction choose karne ka option dunga.
- **2. Dynamic Tool Calling Agents:** Researcher ko static search ke bajaye ReAct tool-use agent banaunga with ArXiv, PubMed, aur Financial Data APIs.

Final High-Impact Closing Pitch for Interview

*"Yeh project sirf ek standard prompt-based chatbot nahi hai — yeh ek **fault-tolerant, deterministic state machine** hai jo asynchronous agent coordination, adversarial verification, hybrid vector retrieval aur universal MCP tooling ko integrate karke publication-grade autonomous research deliver karta hai."*